

1. Escopo e Regras de Negócio (Ideia de Projeto)

Toda a interface foi desenhada para atacar os problemas corporativos que vocês levantaram:

- **Foco no Público-Alvo:** O assistente foi estruturado para resolver o desafio financeiro e operacional do treinamento e integração de estagiários e novos colaboradores.
- **Redução de Gargalos:** O sistema guiado atua para evitar que o funcionário interrompa um superior, reduzindo a perda de produtividade em cadeia e os riscos de retrabalho ao tentar resolver problemas sem orientação.
- **Estratégia de Validação:** A adoção de um ambiente com dados simulados e procedimentos próprios, garantindo que a ineficiência de depender dos dados restritos de uma empresa real não prejudique a aplicação.
- **Categorização do Conteúdo:** As trilhas e consultas do código refletem a necessidade de absorver Manuais de Integração, Procedimentos Operacionais Padrão, Políticas Internas e Guias de Uso de Ferramentas.

2. Comportamento e Interação (Aula 02 — Agentes Inteligentes)

A lógica da interface de treinamento foi pensada com base na taxonomia de agentes:

- **Jornada do Agente:** O programa atua como um Agente Inteligente focado no usuário, mapeando as interações em um ambiente de aprendizado.
- **Treinamento Prático:** O "Modo Simulação" criado no script aplica o conceito de testar ações dentro de um ambiente seguro antes da execução real.
- **O "Cérebro" do Consultor:** A consulta livre foi preparada para, no futuro, ser integrada a um Agente LLM utilizando o padrão *ReAct (Reason + Act)*, permitindo que a IA raciocine sobre os manuais antes de responder à dúvida do usuário.

3. Estruturação da Informação (Aula 05 — Representação do Conhecimento)

A forma como o código armazena e recupera as regras utiliza conceitos de IA Simbólica:

- **Sistemas Baseados em Regras:** As trilhas de treinamento passo a passo comportam-se como Sistemas Especialistas, garantindo que procedimentos exatos (como a triagem de um ticket) sejam seguidos rigorosamente.
- **Arquitetura Híbrida:** A estruturação dos dicionários em Python simula uma Base de Conhecimento.
- **Recuperação Precisa:** A lógica do assistente serve como base para implementar *RAG (Retrieval-Augmented Generation)*, unindo a capacidade de linguagem do LLM com os fatos verificáveis dos manuais simulados para zerar as alucinações.

4. Criação da Empresa Fictícia (Aula 01 — Panorama da IA e Prompts)

A fundação para dar vida a esse projeto envolverá as técnicas iniciais da disciplina:

- **Geração Sintética:** Para popular o código com manuais realistas, utilizaremos técnicas de Engenharia de Prompt, como *Role Prompting* (atribuindo o papel de um gerente de RH ou TI ao modelo).

- **Padronização de Código:** O uso de *Structured Output* (pedindo que o LLM retorne os POPs em JSON, por exemplo) para que os documentos gerados se encaixem perfeitamente nas variáveis do script em Python.